

Do We Still Need GPUs?

Rethinking AI and Scientific Computing on Matrix-Enhanced CPUs

Jack Dongarra,¹ Torsten Hoefer,² Satoshi Matsuoka³

¹University of Tennessee ²ETH Zurich ³RIKEN Center for Computational Science (R-CCS)

CACM revised version — June 19, 2026

Abstract. We examine whether Graphics Processing Units (GPUs) will remain indispensable for artificial intelligence (AI) and high-performance computing (HPC) as central processing units (CPUs) evolve. While GPUs have dominated the landscape due to their massive parallel throughput and high memory bandwidth, modern CPU architectures are increasingly incorporating specialized hardware features like ARM’s SVE/SME and Intel’s AVX/AMX, combined with on-package High-Bandwidth Memory (HBM) and native mixed-precision support (from FP64 down to FP4). We argue that integrating matrix engines directly into the CPU pipeline addresses the core reasons GPUs became necessary, reducing the latency, energy costs, and programming complexities associated with moving data across discrete accelerator interfaces. This CPU-centric approach is highly advantageous for converged workflows that combine AI-style tensor throughput with irregular, branch-heavy scientific logic. Ultimately, we conclude that while GPUs will remain vital for frontier-scale AI training, an evolved, accelerated CPU offers a simpler, more balanced, and highly programmable foundation for the next generation of scientific computing. To put this on a quantitative footing, we summarize a companion empirical study—in preparation for arXiv—that tests the argument on two real Arm CPUs running long-context inference for a trillion-parameter model: the Fugaku A64FX, which has wide vectors and HBM but no matrix engine, and the newly shipping LX2, which adds one. The measurements on the A64FX establish the bandwidth half on deployed hardware, while the same model applied to the LX2’s published specifications indicates the matrix-engine half is met—an asymmetry we keep explicit, since the A64FX results are measured but the LX2 results are projected from its specification and await direct validation. Three issues—the interconnect, FP8 support, and an energy premium tied to memory generation—qualify but do not overturn the conclusion.

The architectural argument

For over ten years, Graphics Processing Units (GPUs) have served as the primary accelerator for artificial intelligence and a cornerstone of modern high-performance computing (HPC). Originally engineered to handle the massive pixel-level parallelism required for complex graphical rendering, the underlying architecture was generalized nearly two decades ago [1]. This evolution transformed GPUs from application-specific hardware into powerful, programmable accelerators capable of executing highly parallel, general-purpose workloads [2]. The following success is easy to understand: GPUs provide massive floating-point throughput using optimized datatypes, excellent efficiency for dense linear algebra, matrix-matrix operations, high memory bandwidth, and many forms of data-parallel computation. Modern AI, especially deep learning, has been shaped around the strengths of GPUs, and many scientific applications have been rewritten to exploit them.

But the dominance of GPUs should not be mistaken for inevitability. GPUs became essential because conventional CPUs lacked sufficient parallel arithmetic capability or memory bandwidth for emerging AI and high-performance computing workloads. If future CPUs include wide vector units, matrix-multiply engines,

large amounts of high-bandwidth memory, efficient low-precision arithmetic, and support for the right datatypes, then the argument changes. In that world, the distinction between a “CPU” and an “accelerator” becomes less important. What matters is not whether the device is called a GPU, but whether it provides the right computational structures, memory system, programming model, and numerical capabilities. We make that architectural case in what follows, and then—because an argument of this kind is only as strong as the hardware that bears it out—summarize a companion empirical study on two real Arm CPUs, one without a matrix engine and one with, that tests it on a demanding long-context AI inference workload; the comprehensive version is in preparation for arXiv.

A modern CPU with ARM SVE and SME [3] or Intel’s AVX and AMX [4], large HBM capacity, support for FP64, FP32, FP16, block-scaled FP8, and FP4 arithmetic, and a matrix-multiply engine begins to address precisely the reasons GPUs became necessary in the first place. Such a processor could offer much of the computational throughput needed for AI while retaining the generality, programmability, and system integration advantages of a CPU. Indeed, this architecture would move the acceleration into the CPU pipeline and thus make it easier to use. For scientific computing, this is especially attractive because many applications require a mixture of dense linear algebra, sparse operations, irregular communication, reductions, control flow, and high-precision arithmetic. GPUs are powerful, but they are not always the ideal match for this broader workload mix.

The central computational kernel in both AI and much of scientific computing processes matrices and vectors, similar to dense linear algebra calculations. In AI, training and inference are dominated by matrix-matrix multiplication, convolution, and attention kernels. Usually, the dimensions are large enough that these operations map extremely well to highly parallel hardware. In scientific computing, dense linear algebra remains critical in eigensolvers, factorizations, simulation codes, optimization, uncertainty quantification, and increasingly in AI-assisted modeling. If a CPU contains vector and matrix engines with high bandwidth and the right datatype support capable of delivering high throughput for these operations, then one of the GPU’s most important historical advantages is diminished.

Memory bandwidth is most important for scientific computing and generative AI inference. The main reason GPUs became so successful is that they offered far more bandwidth than conventional CPU memory systems. Many scientific and AI workloads are not limited only by arithmetic throughput; they are limited by the rate at which data can be supplied to the arithmetic units. Adding large amounts of HBM to a CPU changes this balance. HBM provides the bandwidth needed to feed vector and matrix units, reducing the traditional gap between CPUs and GPUs. A CPU with on-package HBM can avoid the bottleneck of moving data across a separate accelerator interface and can keep computation closer to memory.

ARM’s Scalable Vector Extension, or SVE, provides a vector-length-agnostic programming model that allows software to run across implementations with different vector widths. This is valuable because it avoids hard-coding software to a particular SIMD width. ARM’s Scalable Matrix Extension, or SME, goes further by adding architectural support for matrix operations, including outer-product style computation that is well-suited to dense linear algebra and AI kernels. Other architectures such as Intel’s AVX and AMX support similar acceleration. If these features are implemented with high throughput and paired with sufficient memory bandwidth, then the CPU becomes not merely a scalar control processor but a true numerical engine.

This matters because data movement is often the real cost in modern computing [5]. Moving data between CPU memory and GPU memory adds latency, consumes energy, complicates programming, and forces applications to manage multiple memory spaces. Unified memory models and high-speed interconnects help, but they do not completely remove the complexity. A powerful CPU with HBM and integrated matrix capability can simplify the execution model. Data can remain in a single memory abstraction, managed by the processor, which also handles the operating system, communication, control flow, and numerical kernels. For many applications, this simplicity is a major advantage.

AI workloads also increasingly depend on low-precision arithmetic. Training and inference often use

FP16, BF16, INT8, and, in some cases, even lower-precision formats. GPUs gained an enormous advantage by introducing tensor cores optimized for these formats. But there is nothing inherently GPU-specific about low-precision matrix arithmetic. If a CPU includes native support for FP4, FP16, FP32, FP64, INT8, as well as block-scaled FP8 and FP4, together with a matrix-multiply engine, it can serve many AI workloads directly. Low precision formats are particularly important for inference, where quantized models can deliver high throughput and lower energy consumption. FP16 and BF16-style computation are important for training and mixed-precision algorithms. FP64 remains essential for many scientific applications and for recovering accuracy when lower precision is used.

This mixed-precision capability is especially important for scientific computing. The future of scientific computation is not simply “use lower precision everywhere.” Rather, the right strategy is to use the lowest precision that is safe for each part of the computation, while retaining higher precision where stability, convergence, and reproducibility require it. A CPU that supports multiple precisions natively is well suited to this model. It can perform bulk computation in lower precision, accumulate or correct in higher precision, and use double precision where necessary [6]. This is a natural match for iterative refinement, mixed-precision solvers, AI-accelerated simulations, and algorithms that combine numerical modeling with learned components.

Another reason we may not need GPUs in such a system is programmability. GPUs are very effective, but they often require specialized programming models, careful kernel design, memory hierarchy management, and attention to occupancy, thread divergence in older models, and data placement. Excellent programming systems exist, but the complexity is real. Scientific applications are often large, long-lived codes developed over decades by teams whose primary expertise is in physics, chemistry, engineering, applied mathematics, or climate science, not necessarily GPU kernel optimization. A CPU-centered architecture with strong vector and matrix capabilities can lower the barrier to performance. Existing programming models, compilers, OpenMP, MPI, math libraries, and operating-system services can be extended rather than replaced.

This does not mean that software comes for free. High performance will still require optimized libraries, careful data layout, compiler support, and algorithmic restructuring. But the path may be simpler if the processor remains a general-purpose CPU with accelerator-like features built in. Instead of treating the GPU as a separate device, the programmer sees a powerful CPU with scalable vector and matrix instructions for the right datatypes and high-bandwidth memory. That can make it easier to support irregular workloads, dynamic control flow, adaptive methods, sparse computations, and tightly coupled simulations.

Scientific computing is broader than dense matrix multiplication. Many important applications involve sparse matrices, graph-like data structures, adaptive meshes, particles, multiscale coupling, irregular memory access, global reductions, and communication across distributed systems. GPUs can handle many of these workloads, but efficiency is often harder to achieve than for dense tensor operations. CPUs are traditionally better at latency-sensitive, branch-heavy, and irregular computation. If the CPU also has strong dense linear algebra performance, it can handle both sides of the workload: the regular, compute-intensive kernels and the irregular orchestration around them. This reduces the need to split an application awkwardly between CPU and GPU and removes unnecessary data movement.

A CPU-only approach also has potential system-level advantages. Removing discrete GPUs can reduce system complexity and data movement between CPU and GPU. There may be fewer programming environments, fewer memory spaces, fewer data transfers, fewer points of failure, and potentially simpler scheduling. In large-scale systems, reliability, power, cooling, packaging, and network integration all matter. A homogeneous node design can be easier to manage than a heterogeneous CPU-GPU node, especially if the CPU provides enough arithmetic intensity and memory bandwidth to meet the application’s needs.

There is also a strong argument around balance. A well-designed scientific computer is not defined by peak flop/s alone. It must balance arithmetic performance, memory bandwidth, memory capacity, network bandwidth, latency, energy efficiency, software maturity, and numerical precision. GPUs often deliver

extraordinary peak performance, but not all applications can use that peak. A CPU with matrix hardware and HBM may offer a better balance for many scientific workloads, even if its theoretical peak is lower than that of a GPU-heavy node. For real applications, time to solution, energy to solution, and ease of programming matter more than advertised peak performance.

For AI, the case is more nuanced. Large-scale training of frontier models has been built around GPUs and GPU-like accelerators because they offer enormous dense tensor throughput and energy efficiency as well as a mature software ecosystem. Replacing GPUs completely for the largest models would require CPU matrix engines to achieve comparable efficiency and throughput, memory bandwidth, communication performance, and software support. That is a high bar. But many AI workloads are not frontier-scale training runs. Inference, fine-tuning, scientific machine learning, surrogate modeling, graph neural networks, sparse models, and AI embedded inside simulation workflows may run very effectively on a CPU with matrix extensions and HBM. In these cases, integration and data locality may matter as much as raw tensor throughput.

The strongest argument, then, is not that GPUs are useless. They are not. GPUs are extremely powerful and will remain important. The argument is that GPUs are not fundamentally required if the CPU evolves to include the architectural features that made GPUs attractive. A CPU with SVE/AVX, SME/AMX, HBM, multiple precision formats, and matrix-multiply capability is no longer a conventional CPU. It is a general-purpose processor with accelerator-class numerical machinery.

This shift could be especially important for the convergence of AI and scientific computing. Future scientific applications will not simply run simulations or train neural networks separately. They will combine simulation, data assimilation, optimization, uncertainty quantification, and machine learning in tightly coupled workflows. These workflows need both AI-style tensor throughput and traditional HPC capabilities: MPI communication, double precision, sparse solvers, adaptive algorithms, file I/O, and complex control logic. A CPU with integrated matrix acceleration may be a cleaner platform for this convergence than a system that requires constant movement between a host CPU and a discrete GPU.

In this sense, the question should not be “Do we need GPUs?” but rather “What architectural features do we need?” We need high memory bandwidth. We need efficient dense linear algebra. We need support for FP64, FP32, FP16, and INT8. We need scalable vector and matrix operations. We need strong compilers and libraries. We need efficient communication and a balanced memory hierarchy. We need a programming model that scientists can use productively. If a CPU provides these features, then many of the reasons for using GPUs disappear.

The future may therefore be less about GPUs versus CPUs and more about convergence. GPUs have taught the computing community what high-throughput numerical hardware should look like. CPUs are now absorbing some of those lessons: wider vectors, matrix engines, on-package HBM, lower precision arithmetic, and tighter integration with the memory system. If these trends continue, the CPU can become a much more capable platform for both AI and scientific computing.

Putting the case to the test: a comprehensive empirical companion

The argument so far is conceptual, but an architectural claim is only as strong as the hardware that bears it out. We have therefore carried out a detailed empirical study to ask whether it holds quantitatively on machines that exist today, and we summarize that study here; a comprehensive version—with the full performance models, measurements, system-sizing, power accounting, and methodology—is in preparation for publication on arXiv [7].

The workload, and two CPUs that bracket the thesis. A fair test needs a workload that exercises both halves of the argument—bandwidth and matrix throughput—and hardware that isolates the contribution of each. For the workload we fix production long-context inference for a trillion-parameter Mixture-of-Experts model (Kimi-K2, which uses Multi-head Latent Attention) at a 256K-token context, with the full

modern acceleration stack enabled from the outset: INT4 quantization-aware weights, INT4/INT8 KV-cache quantization, DeepSeek Sparse Attention, and EAGLE-3 speculative decoding. These accelerations are model-architectural and orthogonal to the hardware question, which is exactly why they belong in the baseline—they shrink the work before the architecture is asked to do it, and a fair comparison must assume a competent software stack on both sides. We are explicit about the status of each: MLA and INT4 quantization-aware weights are native to K2, EAGLE-3 is treated as an integration multiplier, and DSA is a sensitivity projected from DeepSeek-V3.2 rather than a result co-validated on K2. For hardware we use two real Arm CPUs that differ in precisely the feature under test. The A64FX of the Fugaku supercomputer is a deployed instance of the accelerated CPU *minus* the matrix engine—wide SVE vectors and 32 GB of on-package HBM at about 1 TB/s per node, but no SME—which makes it a clean control for measuring what a matrix engine must contribute. The newly shipping Armv9 LX2 of the all-CPU “LineShine” system is a deployed instance *with* the matrix engine: SVE and SME on every core, 32 GB of HBM at 4 TB/s, and 240 TF of BF16 and 960 TOPS of INT8 matrix throughput per socket. Together they turn a conjecture about future CPUs into a measurement-plus-projection on present ones: measured on the A64FX for the bandwidth side, and evaluated against published LX2 specifications for the matrix-engine side.

Why AI inference is the test. The choice of an AI workload, rather than a simulation kernel, is deliberate. For the memory-bound, sparse, and graph-structured kernels that dominate much of traditional simulation, the thesis is no longer a conjecture, and Fugaku is the clearest proof point: the same matrix-engine-poor A64FX was the world’s #1 system on the Top500 from 2020 to 2022 and simultaneously led HPCG and Graph500—the memory-bound and sparse benchmarks built from the kernels real simulation actually runs. On exactly those codes a bandwidth-rich CPU already matches or sometimes beats GPU machines of higher peak FLOP/s; the larger body of GPU-accelerated *dense* simulation is the part that is not settled. The objection usually raised here is the Green500, which appears to show GPUs as far more power-efficient; but it answers a narrower question—energy efficiency on HPL, a *dense FP64 matrix-multiply* benchmark—and a vector CPU without a matrix engine is disadvantaged on that particular workload, which is precisely the gap SME-style engines are meant to close. That disadvantage does not arise for the memory-bound kernels that dominate simulation, where Fugaku already led. The one regime in which GPUs are still *perceived* to hold a decisive advantage, and in which the question is therefore genuinely open, is modern AI. That is why the companion takes AI inference as its target.

Decode: the bandwidth half already holds. Generative inference has two phases with opposite hardware demands, and they test the two halves of the argument separately. Decode—the autoregressive generation of output tokens, one at a time—must stream the model’s active weights and the entire key/value (KV) cache from memory for every token it produces, so it is bound by memory bandwidth, not arithmetic. The study finds K2 decode roughly 80% bound by HBM bandwidth and only about 1% bound by floating-point throughput; the matrix engine is essentially idle, and aggregate memory bandwidth is what sets the token rate. Decode performance therefore scales with the total HBM bandwidth a cluster can bring to bear, and on that measure the CPU is not a future prospect but already competitive: roughly 48 A64FX nodes deliver the same K2-256K decode throughput as a current GPU node (a GB200 NVL4), because their aggregate HBM bandwidth is comparable. For the phase that dominates the token-generation cost of long-context serving, in other words, the bandwidth claim of this article is confirmed on hardware shipping today—and it is confirmed on the A64FX, which has no matrix engine at all.

Prefill: the matrix engine, and how much it must deliver. Prefill—the initial, parallel processing of the input prompt—is the opposite: it is compute-bound and rich in dense matrix work, and it is where a matrix engine earns its place. The central result is that “parity” on prefill is not a single number but a range that depends entirely on which GPU one measures against. The A64FX, with no matrix engine, trails a top GPU by about $47\times$ at peak on prefill. Closing that gap requires a per-node matrix throughput somewhere between about 80 TF/node—if the baseline is a sensibly balanced GPU configuration with sparse attention

enabled, the regime most real deployments occupy—and more than 750 TF/node if the baseline is a maximal, dense, fastest-GPU configuration running without such savings. The lower, feasible target amounts to roughly a $6\times$ per-node uplift over the A64FX, a reasonable thing to ask of an added matrix engine; the maximal target is not. The decisive point is that the LX2 already clears the feasible bar: its 240 TF of BF16 is about $3\times$ the balanced target, and its 960 TOPS of INT8 exceeds even the maximal one. On the LX2’s *published specifications*, then, the matrix-engine half of the argument is met—conditionally, because the right baseline matters—and the engine it calls for is no longer hypothetical, because it ships on every core of the LX2; what remains is to confirm these figures on the machine itself, since unlike the A64FX decode result they are projected, not yet measured.

Three issues qualify the result without overturning it: the behaviour of the interconnect, the energy premium, and the precision format. Each is worth more than a sentence, because each is where a careless reading would either overstate or dismiss the CPU’s case.

The interconnect, and why it decides between pipeline and tensor parallelism. The results above are per node, but serving a trillion-parameter model requires hundreds of nodes working together, and how they are made to cooperate is the single most consequential system-level variable. A model too large for one node is split across many in one of two ways. *Tensor parallelism* (TP) shards each individual layer across nodes and, for every layer, performs an all-reduce to recombine the partial results—a collective communication whose cost is set by the achieved all-reduce bandwidth of the interconnect, and whose data volume during long-context prefill is large. *Pipeline parallelism* (PP) instead assigns whole groups of consecutive layers to different nodes and passes activations from one stage to the next—a point-to-point transfer that moves far less data and needs no collective at all. Which strategy is better depends directly on the fabric, and this is the crux. On a torus interconnect such as Fugaku’s Tofu-D, the achieved all-reduce bandwidth is modest (about 6 GB/s in practice), so long-context TP prefill becomes communication-bound: the network, not the processor, sets the time, and the matrix engine sits waiting. The remedy is to run prefill with pipeline parallelism instead. Because PP exchanges only stage-boundary activations, point-to-point, it sidesteps the very collective the torus handles poorly, and prefill becomes compute-bound again. When the interconnect is strong—an Ethernet-class CPU fabric such as the LX2’s, which sustains roughly an order of magnitude more all-reduce bandwidth—TP no longer bottlenecks, and the simpler, uniform wide-TP configuration becomes the better choice for both phases. The practical conclusion is that a CPU-based system is not locked into one topology: it can be built either way, matching the parallelization strategy to the network—leaning on pipeline parallelism where collective bandwidth is scarce, and on tensor parallelism where it is plentiful. The interconnect is thus a design choice, not a fixed liability; paired with an adequate fabric, a CPU machine removes the communication wall that a weak torus would otherwise impose, and even on a weak fabric the right parallelization recovers most of the loss.

Energy: a real premium, but a generation gap rather than an architectural one. On power the honest verdict is that the all-CPU machine is competitive but behind, and it is worth being precise about how far behind, and why. Sized for this workload, an all-LX2 machine draws roughly 1.75 to 2.7 times the per-user power of a fleet of the newest HBM3e GPUs. The range is not vagueness but two distinct ways of organizing the machine: at the low end, a co-located configuration in which prefill is absorbed in the spare matrix capacity of the decode sockets; at the high end, a role-split configuration that provisions a separate pool of prefill sockets and therefore more silicon. The co-located figure is a throughput upper bound: prefill also draws HBM bandwidth and scheduling slots from a decode phase that is itself $\sim 80\%$ bandwidth-bound, so it holds only under admission control that caps prefill bursts within the decode latency budget, and role-split is the safer service-quality point. An engine-less hybrid—an A64FX decode tier bolted to a separate, GPU-free prefill plant—is worse still, at roughly 3 to 4 times. The important question is where the premium comes from, and the answer is mostly *not* the architecture. Decode dominates the energy because it dominates the token stream, and decode is bandwidth-bound, so its per-user energy is governed largely by memory power—and

the LX2 carries an older HBM2-class memory generation, whereas the newest GPUs use HBM3e, which simply delivers more bandwidth per watt. Most of the remainder is the missing FP8 path, which would let prefill run in a denser low-precision format and so finish in less energy. Both of these are generational rather than fundamental: an HBM3e-class CPU with an FP8-capable matrix engine—both of which are on public roadmaps—would erase most of the difference. The premium is also robust in the other direction, which matters for honesty about the comparison. Applying a common facility overhead for cooling and power distribution to both machines preserves the ratio rather than inflating it; and because a self-contained CPU socket carries far less external switching than an NVLink-coupled GPU node, a fuller system-level accounting tends to narrow the premium, not widen it. The energy story, in short, is that the CPU pays a measurable but shrinking tax that tracks memory technology and precision support rather than the architecture itself.

Precision, and a research direction we are pursuing. The third issue is precision, which separates into a near-term empirical question and a longer-term research direction that we find promising but still unsettled. The near-term question is concrete: today’s shipping CPU supports INT8 and BF16 but not FP8, and it remains to be determined whether INT8 offers a fine enough quantization grid for accurate long-context prefill or whether FP8—expected in a future generation of SME—will be required.

The longer-term question is whether low-precision matrix multiplication can be used as a common computational primitive across both AI and scientific computing. Techniques based on numerical decomposition [8] can express a high-precision matrix product as a sequence of low-precision matrix multiplications followed by suitable accumulation and reconstruction. In principle, the same FP8 or INT8 matrix engine used for neural-network inference could therefore accelerate selected FP32- and FP64-accuracy numerical kernels.

Such an approach could simplify the hardware stack: a single high-throughput matrix engine might support arithmetic ranging from low-precision AI workloads to carefully constructed high-precision computations. This possibility is particularly attractive for a future bandwidth-oriented CPU whose memory bandwidth reaches tens of TB/s while its native FP64 throughput remains comparatively modest. For suitable compute-intensive kernels, low-precision matrix engines could provide additional arithmetic capability without requiring a correspondingly large conventional FP64 unit.

We regard this as a research hypothesis rather than an established result. Its usefulness will depend on numerical conditioning, error control, decomposition and accumulation costs, data movement, and the efficiency attainable on real hardware. It is also likely to apply most naturally to dense matrix operations rather than uniformly across sparse, irregular, communication-intensive, or general-purpose scientific workloads. Determining where the approach is accurate and advantageous across representative scientific kernels is an important part of the follow-on research we intend to conduct.

On the current LX2, which supports modest INT8 relative to its strong FP64, and not FP8, native FP64 remains the more appropriate path outside dense matrix multiplication. The research question concerns the capabilities of future matrix-enabled CPUs, not a claim about the present chip.

Conclusion

Returning to the question of the title: we may not need GPUs for many AI and scientific workloads if we have a CPU with SVE and SME, large amounts of HBM, native support for FP64, FP32, FP16, and INT8, and a high-throughput matrix-multiply engine. Such a processor attacks the very limitations that led to GPU adoption—insufficient parallelism, insufficient memory bandwidth, and the lack of low-precision tensor performance—while preserving the advantages of CPUs: generality, programmability, strong control flow, mature system software, and suitability for irregular scientific workloads. The empirical companion turns this from an aspiration into a measured-and-modeled claim for the inference and converged-simulation regimes. The bandwidth half of the case already holds on hardware shipping today, demonstrated on a CPU with no matrix engine at all; the matrix-engine half is met by a CPU now shipping, on its published

specifications and pending direct measurement; and the issues that remain are bounded and addressable—an interconnect that a CPU system can accommodate by choosing pipeline or tensor parallelism to suit its fabric, an FP8 mode already specified for the next matrix-engine generation, and an energy premium that tracks memory generation rather than architecture. The distance that remains to the GPU frontier is therefore a matter of engineering and timing, which makes the decision to build a GPU-free machine, for now, largely one of sovereignty and supply rather than of absolute energy efficiency, on current estimates. The largest non-hardware gap remains software maturity: GPU libraries and serving stacks are deeply optimized today, and CPU matrix engines will need comparable compiler, kernel, and runtime support before the architectural promise is fully realized. GPUs will remain valuable, particularly for the largest dense AI training problems, but they are not the only path forward. A balanced CPU with integrated matrix and memory capabilities can provide a simpler, more unified, and highly effective foundation for the next generation of AI-enabled scientific computing—and the full measurements, models, system-sizing, and analysis behind this summary are in preparation for publication on arXiv.

Disclosure of AI assistance

The authors used Anthropic’s Claude (Opus 4.8) as an interactive tool for drafting, consistency checking, and arithmetic review, and thank OpenAI’s ChatGPT (GPT-5.5) for critical and constructive review comments on successive drafts. All hardware parameters, assumptions, analyses, and conclusions were reviewed, corrected, and approved by the authors, who take full responsibility for the work, including any errors of fact or judgment. The comprehensive companion study, in preparation for arXiv, gives a fuller account of the methodology; direct hardware measurement of the LX2 projections is the explicit next step.

References

- [1] J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone, and J. C. Phillips. GPU Computing. *Proceedings of the IEEE*, 96(5):879–899, 2008.
- [2] J. Nickolls and W. J. Dally. The GPU Computing Era. *IEEE Micro*, 30(2):56–69, 2010.
- [3] Arm. *Arm Architecture Reference Manual for A-profile architecture*, chapter: The Scalable Matrix Extension (SME), for Armv9-A. Arm Developer.
- [4] I. Sodhi et al. Intel AMX: Advanced Matrix Extensions for Xeon Scalable Processors. *IEEE Hot Chips 34 Symposium*, 2022.
- [5] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefer. Data Movement Is All You Need: A Case Study on Optimizing Transformers. *Proceedings of the Fourth Conference on Machine Learning and Systems (MLSys)*, 2021.
- [6] A. Haidar, S. Tomov, S. Donfack, R. Pranesh, N. J. Higham, and J. Dongarra. Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers. *IEEE Transactions on Parallel and Distributed Systems*, 31(12):2796–2808, 2020.
- [7] J. Dongarra, T. Hoefer, and S. Matsuoka. Do We Still Need GPUs? A Concrete A64FX/LX2 Case Study. In preparation; to appear on arXiv, 2026.
- [8] S. Matsuoka. FP8 is All You Need (Part 1): Debunking Hardware FP64 as the HPC Holy Grail. arXiv:2606.06510, 2026.